DEEP LEARNING ENGINEER

Technical Work Package & Implementation Blueprint

Remaining Useful Life (RUL) Prediction for Jet Engines
NASA C-MAPSS Dataset

Temporal analysis · sequence engineering · LSTM regression · hyperparameter tuning · uncertainty quantification · interval evaluation · classical-model comparison

PINE GREEN / GRAPHITE / WHITE
Research-grade methodology • Reproducible implementation • Defense-ready documentation

A detailed execution plan for the third team member, aligned with the supplied RUL task division document

Integrity note: This guide provides a rigorous methodology and reference implementation. It does not invent experimental scores. Performance tables must be populated only after running the code on the agreed dataset split.

01 | Mission and Definition of Done

The Deep Learning Engineer owns the temporal modelling work package: analyze cycle-by-cycle engine histories, build fixed-length sequences, train an LSTM model to estimate Remaining Useful Life (RUL), tune it without leakage, quantify predictive uncertainty, and compare the resulting model against the team's classical baselines. The implementation must follow the team's shared target definition, split strategy, and evaluation functions.

The supplied task division explicitly assigns temporal-structure analysis, temporal input preparation, LSTM implementation and tuning, uncertainty estimation such as Conformal Prediction, Coverage and interval Width, comparison against classical models, and documentation to this role.

Definition of done

- Runnable end-to-end pipeline from prepared data to saved predictions and metrics.
- Leakage-aware validation: transformations are fitted on training data only and engine-level grouping is respected.
- Reproducibility: seeds, configuration, split IDs, package versions, checkpoints, and experiment logs are saved.
- Comparable evaluation: LSTM and classical models use the same target convention, test engines, endpoints, and metric functions.
- Uncertainty audit: calibration protocol, nominal level, empirical Coverage, and Mean Interval Width are reported together.
- Clear limitations: the report distinguishes measured results from assumptions and proposed settings.

02 | Problem and Dataset Contract

NASA C-MAPSS is a simulated turbofan-engine degradation benchmark. Typical records contain an engine/unit identifier, operating cycle, operating settings, and sensor measurements. At a chosen cycle, the objective is to estimate the number of cycles remaining until failure. Confirm the exact subset (FD001–FD004), file version, columns, target convention, and available test labels with the team before modelling.

Decision	Required action	Record in experiment log
Dataset subset	Name the selected C-MAPSS subset; do not silently mix variants.	Subset, file paths, row/engine counts
Target convention	Use the shared RUL calculation and cap policy consistently.	Formula, cap value or "uncapped"
Prediction endpoint	Define whether each input predicts RUL at its final observed cycle.	Endpoint convention
Evaluation unit	Usually one prediction per test engine's final observed window.	Engine IDs and label source
Shared evaluator	Reuse the project's agreed metrics and post-processing.	Evaluator version / function

Do not fabricate numeric findings. If the official test RUL labels are supplied separately, join them according to the dataset protocol and keep them out of training and model selection.

03 | Temporal Data Audit

Before modelling, verify the chronology and identify patterns that affect sequence construction. This audit should be a saved artifact, not an informal glance at the data.

Required analyses

- Sort by engine ID and cycle; check strictly increasing cycles and duplicate engine-cycle pairs.
- Summarize trajectory length per engine and plot the distribution. Explain unusually short or long trajectories.
- Plot several sensor channels against cycle for multiple engines; inspect drift, abrupt changes, flat channels, and noise.
- Summarize sensor mean, standard deviation, minimum, maximum, missingness, and constant/near-constant channels.
- Inspect correlations and operating-setting variation; note potential regime effects.
- Confirm which training trajectories reach failure and how test histories differ.
- Check missing values, infinities, duplicate rows, and unexpected schema changes.

Minimum outputs

- Dataset profile table: rows, engines, cycle range, settings, sensors, missing values, duplicate count.
- Engine trajectory-length histogram and at least three sensor-versus-cycle plots.
- Sensor summary/correlation outputs and a short written interpretation.



Figure 1. End-to-end modelling workflow. Learned preprocessing must be fitted on training data only.

04 | Temporal Input Engineering

An LSTM consumes ordered sequences. For each engine, create a sliding window of L consecutive cycles. With F input features, the tensor shape is (N, L, F). Each window is labelled with the RUL value at the final cycle represented in that window.

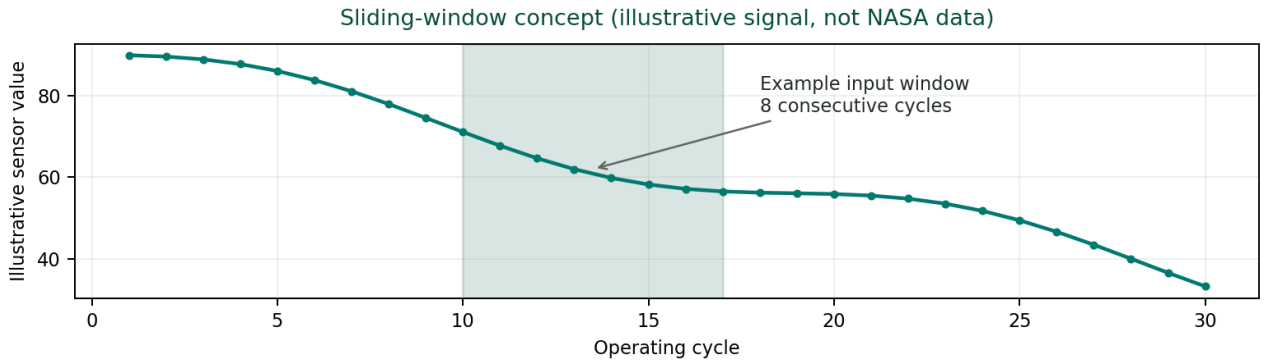


Figure 2. Conceptual sliding window. The signal is illustrative, not a NASA sensor trace.

Window construction rules

- Group by engine first, sort by cycle, and build windows independently within each engine. Never cross engine boundaries.
- Choose a primary sequence length (e.g., 30 cycles) and test a small predeclared set such as 20, 30, and 50.
- Use a documented stride; stride 1 is a reasonable starting point for training windows.
- Assign the target for the final cycle in each window—not the first cycle or the next cycle unless the task definition explicitly says otherwise.
- Split engines before generating train/validation windows for unseen-engine generalization. Overlapping windows are highly dependent.
- For test evaluation, use the final L observed cycles per engine when enough cycles exist. Define padding/truncation for shorter histories.
- Do not include future sensor rows, failure-cycle information, or future-derived statistics as input features.

Scaling and feature handling

- Fit feature scalers using training-engine rows only; freeze and apply them to validation, calibration, and test rows.
- If target scaling is used, inverse-transform predictions before reporting errors in cycles.
- Do not treat engine ID as a predictive sensor. Evaluate raw cycle as a feature only if justified and documented.
- Persist the exact feature list and column order with the scaler and model.

Tensor contract

```
X_train: float32 [N_train_windows, sequence_length, n_features]
y_train: float32 [N_train_windows]       # RUL at window endpoint
X_valid: float32 [N_valid_windows, sequence_length, n_features]
y_valid: float32 [N_valid_windows]
X_test:  float32 [N_test_engines, sequence_length, n_features]
engine_id_test: [N_test_engines]          # metadata, not a model feature
```

Acceptance tests

- Every window has exactly L rows and strictly increasing cycles.
- Every window contains one and only one engine ID.
- The endpoint target matches the target label at that cycle.
- The number of final test predictions equals the number of engines evaluated under the declared protocol.
- Manually verify at least one engine's final window and target.

05 | Split Strategy and Leakage Prevention

Randomly splitting overlapping windows can produce optimistic results because nearly identical sequences from the same engine may appear in both training and validation. Use engine-level groups for model selection whenever the intended deployment is generalization to unseen engines.

Stage	Recommended rule	Purpose
Training	Training engines only; create windows within each engine.	Fit model weights

Stage	Recommended rule	Purpose
Validation	Distinct held-out training engines.	Select architecture and hyperparameters
Calibration	Held-out calibration engines, not used to fit the final model.	Estimate conformal residual quantile
Final test	Official untouched test engines or team-agreed test split.	Final generalization estimate

Preserve the official train/test protocol. If a separate calibration split is infeasible, use a defensible cross-fitting design and document its assumptions. Never tune hyperparameters or calibrate intervals on the final test labels.

Leakage checklist

- No engine overlap between training and validation groups when testing unseen-engine generalization.
- Scaler, imputer, feature selector, and all learned transformations are fitted only within training folds.
- Hyperparameters are selected using validation metrics—not final test metrics.
- Calibration scores are generated on data not used to fit the final model, or via a valid cross-fitting procedure.
- Repeated peeking at final test results is avoided and never used for model selection.

06 | LSTM Architecture

Start with a compact architecture that can be understood, reproduced, and tuned. Complexity should be added only when validation evidence supports it.



Figure 3. Reference LSTM architecture; values are starting settings, not claimed optima.

Component	Starting configuration	Rationale / alternatives
Input	(L, F) sequence	Ordered multivariate sensor history
LSTM layer	64 hidden units; final state	Learns sequential representation
Regularization	Dropout 0.15	Try 0.0–0.3 if overfitting
Dense head	32 units, ReLU → 1 output	Continuous RUL regression
Loss	Mean Squared Error	Report MAE/RMSE separately
Optimizer	Adam, learning rate 1e-3	Try 3e-4 if unstable
Batch size	64	Compare 32 / 64 / 128
Training	Max 100 epochs; early stopping patience 10	Restore best validation weights
Seed	Fixed and recorded	Repeat selected run if feasible

Reference implementation (TensorFlow / Keras)

```
import tensorflow as tf
from tensorflow import keras
from tensorflow.keras import layers

def build_lstm(sequence_length, n_features, units=64,
               dropout=0.15, learning_rate=1e-3):
    inputs = keras.Input(shape=(sequence_length, n_features))
    x = layers.LSTM(units, return_sequences=False)(inputs)
    x = layers.Dropout(dropout)(x)
    x = layers.Dense(32, activation="relu")(x)
    outputs = layers.Dense(1, name="rul_prediction")(x)
    model = keras.Model(inputs, outputs, name="rul_lstm")
    model.compile(
        optimizer=keras.optimizers.Adam(learning_rate=learning_rate),
        loss="mse",
        metrics=[keras.metrics.MeanAbsoluteError(name="mae")]
    )
    return model

callbacks = [
    keras.callbacks.EarlyStopping(monitor="val_loss", patience=10,
                                  restore_best_weights=True),
    keras.callbacks.ReduceLROnPlateau(monitor="val_loss", factor=0.5,
                                      patience=4, min_lr=1e-5)
]
# model.fit(X_train, y_train, validation_data=(X_valid, y_valid),
#           epochs=100, batch_size=64, callbacks=callbacks)
```

This is a reference template. Confirm tensor dimensions, framework version, deterministic settings, and the team's target convention before running it.

Controlled tuning plan

Parameter	Candidates	Selection rule
Sequence length	20, 30, 50	Validation RMSE/MAE and stability
LSTM units	32, 64, 128	Prefer smaller model if scores are similar
Dropout	0.0, 0.1, 0.2, 0.3	Check train–validation gap
Learning rate	1e-3, 3e-4	Monitor convergence and instability
Batch size	32, 64, 128	Balance performance and runtime
Dense width	16, 32, 64	Increase only with validation evidence
Loss	MSE; optional Huber experiment	Choose before final test

Tune in stages: establish a baseline; tune sequence length and units; tune regularization and learning rate; select the configuration using validation data; then rerun the chosen setup with a fixed seed and, if feasible, multiple seeds. Record every experiment in CSV/JSON.

07 | Training Diagnostics and Point Metrics

A successful training run is not enough. Inspect convergence, overfitting, prediction ranges, and error patterns. A single average metric can hide systematic failure on particular engines or RUL ranges.

Required diagnostics

- Training and validation loss versus epoch, with best epoch identified.
- Train and validation MAE/RMSE to quantify the generalization gap.
- Predicted-versus-true scatter plot and residual histogram.
- Error by true-RUL bands and by engine where sample size permits.
- Predicted RUL range, non-finite values, and negative predictions. If clipping to zero is used, document it and evaluate its effect.
- Training time and parameter count if compute cost matters to model selection.

Metric definitions

Metric	Definition / interpretation	Reporting note
MAE	Mean(|y − ŷ|)	Lower is better; usually cycles
RMSE	sqrt(Mean((y − ŷ)²))	Penalizes large errors more strongly
NASA Score	Asymmetric exponential penalty used by some RUL benchmarks	Use the exact shared formula and sign convention
Coverage	Fraction of true values inside intervals	Compare empirical with nominal level
Mean Width	Mean(upper − lower)	Lower is sharper only when coverage is acceptable

The shared evaluation utility is the source of truth for metrics. Do not compare values generated by different implementations or post-processing rules.

08 | Predictive Uncertainty with Conformal Prediction

A point estimate does not show how uncertain a prediction may be. Split Conformal Prediction can create intervals with target marginal coverage under exchangeability assumptions. In engine trajectories, adjacent windows are dependent; therefore, calibration must respect the unit of deployment and evaluation as closely as possible.

Recommended first implementation

- Fit the selected LSTM using training data only.
- Predict the held-out calibration set using a calibration unit consistent with deployment (recommended: one final endpoint prediction per engine).
- Compute nonconformity scores $s_i = |y_i - \hat{y}_i|$.
- For nominal coverage $1-\alpha$, use the finite-sample corrected rank $k = \text{ceil}((n_{cal} + 1)(1-\alpha))$; implement the quantile convention carefully.
- Let q be the calibration quantile. For point prediction ŷ, output [ŷ − q, ŷ + q].
- Optionally clip the lower bound at zero because RUL is nonnegative, but disclose that clipping changes interval geometry and may affect empirical coverage.
- Evaluate once on untouched test data and report Coverage and Mean Width together.

Core formulas

```
Coverage = (1 / n) * sum_i I(lower_i <= y_i <= upper_i)
Mean Width = (1 / n) * sum_i (upper_i - lower_i)
Absolute residual score: s_i = abs(y_i - yhat_i)
Nominal coverage: 1 - alpha (e.g., 0.90 when alpha = 0.10)
Split-conformal corrected rank: k = ceil((n_cal + 1) * (1 - alpha))
```

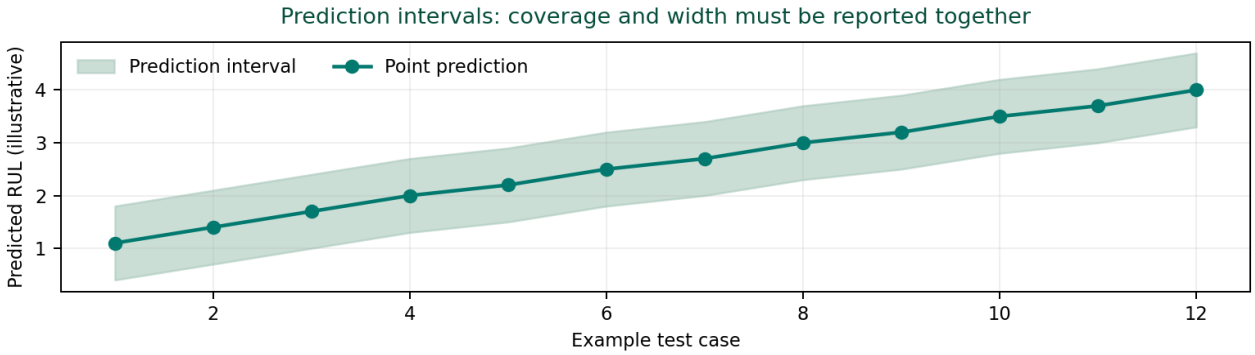


Figure 4. Illustrative point predictions and intervals, not experimental output.

Interpretation and caveats

- Coverage below nominal suggests under-coverage on this test sample; inspect calibration size, shift, and split design.
- Wide intervals can obtain high coverage but be uninformative; always report width beside coverage.
- A nominal 90% marginal guarantee is not a claim that each individual prediction has a 90% conditional probability of being correct.
- Standard split-conformal guarantees rely on exchangeability. Engine dependence and distribution shift may weaken naive row-level assumptions. State this limitation plainly.
- If feasible, compare 80%, 90%, and 95% intervals, with nominal levels chosen before examining test results.

09 | Classical-Model Comparison

The LSTM should be compared against the same models maintained by the other team members. A more complex model is not automatically better; any improvement must be demonstrated under an identical evaluation protocol.

Model	MAE ↓	RMSE ↓	NASA Score* ↓	90% Coverage	Mean Width
Naive / baseline	—	—	—	—	—
Ridge Regression	—	—	—	—	—
LightGBM (if available)	—	—	—	—	—
LSTM	—	—	—	—	—

*Populate with measured values only. Confirm the NASA Score convention used by the shared evaluator; implementations can differ in presentation. Dashes are intentional placeholders, not zero scores.

Fair comparison rules

- Same train/validation/test engine split and RUL target convention.
- Same test endpoint protocol and evaluated engines.
- Same metric functions, target units, and post-processing rules.
- Record runtime and model size if operational efficiency is relevant.
- Discuss whether any LSTM gain justifies added complexity and variance. Report honestly if Ridge or LightGBM performs better.

10 | Code Organization and Artifacts

Keep the modelling component modular so the team can integrate it without inheriting a monolithic notebook. A suggested structure follows; align names with the repository's existing conventions.

```
project/
  configs/
    lstm_baseline.yaml
  src/
    sequence_builder.py       # grouped sorting, windows, endpoint targets
    model_lstm.py             # model factory
    train_lstm.py             # fit, callbacks, checkpointing
    predict_lstm.py           # deterministic inference
    conformal.py              # calibration quantile and intervals
    evaluate_dl.py             # shared metrics and diagnostics
  outputs/
    metrics_lstm.json
    predictions_lstm.csv
    calibration_summary.json
    experiment_log.csv
    figures/
  models/
    best_lstm.keras
    feature_scaler.joblib
  reports/
    deep_learning_methodology.md
```

Minimum prediction schema

Column	Meaning
unit_id	Engine identifier for joining/auditing; never a model feature
true_rul	Ground truth for evaluation only where available
predicted_rul	Point prediction in original RUL units
lower_90	Lower bound of nominal 90% interval
upper_90	Upper bound of nominal 90% interval
sequence_length	Window length used for this prediction
model_version	Configuration/checkpoint identifier

Reproducibility checklist

- Record Python, NumPy, and framework versions; set seeds and note any GPU nondeterminism.
- Save configuration with feature columns, sequence length, architecture, split IDs, target convention, and evaluation settings.
- Save best checkpoint based on validation performance—not test performance.
- Save training history, predictions, metrics, calibration residuals/quantile, and plots.

- Log runtime and exact command used; fail loudly on invalid columns, shapes, or non-finite values.

11 | Deliverables and Acceptance Criteria

Deliverable	Acceptance criteria
Temporal audit	Dataset profile, trajectory-length chart, sensor trends, documented observations.
Sequence builder	Tests for ordering, window length, engine boundaries, endpoint targets.
LSTM implementation	Modular model factory; documented input/output shapes; end-to-end run.
Tuning log	Candidate configurations, validation metrics, selected configuration rationale.
Evaluation package	Predictions and MAE/RMSE/NASA Score from shared evaluator.
Uncertainty package	Calibration protocol, quantile, intervals, Coverage and Mean Width.
Model comparison	Baseline, Ridge, LightGBM if available, and LSTM on identical test protocol.
Technical report	Architecture, settings, results, limitations, and reproduction instructions.
Presentation	Workflow diagram, metric comparison, uncertainty chart, concise conclusion.

12 | Work Plan and Review Gates

Phase	Work	Exit gate
1. Contract	Confirm subset, target, split IDs, metrics, output schema.	Team signs off
2. Audit	Profile trajectories, inspect sensors, decide window policy.	Plots/tables reviewed
3. Baseline	Build windows and train initial LSTM.	Tests and end-to-end run pass
4. Tuning	Run predefined candidates on validation engines.	Best config selected without test leakage
5. Uncertainty	Calibrate intervals and calculate metrics.	Coverage/width reproducible
6. Integration	Use shared evaluator and compare classical models.	Protocol consistency checked
7. Finalization	Write report, save artifacts, prepare defense.	Teammate can reproduce

13 | Common Failure Modes

Failure	Why it matters	Prevention
Random window split	Overlapping windows leak engine-specific patterns.	Split by engine before windowing.
Scaler fit on all data	Validation/test distribution leaks into preprocessing.	Fit transformations on training only.
Misaligned target	Model predicts RUL for wrong cycle.	Label with window endpoint target.
Cross-engine window	Sequence mixes unrelated trajectories.	Group and window per engine.
Test-set tuning	Reported test score becomes optimistic.	Freeze design after validation.
Invalid calibration	Dependent/reused data undermines uncertainty claims.	Separate calibration and match evaluation unit.
Coverage alone	Wide intervals can look artificially good.	Report Coverage and Mean Width together.
Inconsistent comparison	Model ranking is not meaningful.	Share target, split, endpoint, evaluator.

14 | Defense Questions and Model Answers

- Why use an LSTM? It can learn temporal dependencies in ordered sensor histories, but it is not guaranteed to outperform tree-based or linear models.
- Why sliding windows? They create fixed-length histories while preserving order.
- Why split by engine? Overlapping windows from one engine are dependent; grouping reduces leakage and tests unseen-engine generalization.
- What does the model predict? Continuous RUL at the final cycle represented by the input sequence.
- Why fit the scaler only on training data? To prevent information from validation/test distributions influencing learned preprocessing.
- What is early stopping? It stops training when validation performance stops improving and restores the best observed weights.
- What is conformal prediction? A calibration procedure that constructs prediction intervals with marginal coverage under its assumptions.
- What is Coverage? The observed proportion of true RUL values inside the intervals.
- Why report Width? Coverage alone rewards wide intervals; width measures sharpness and must be interpreted jointly.
- Does 90% Coverage mean each prediction has a 90% chance of being correct? No. The usual guarantee is marginal under assumptions, not an individual conditional guarantee.
- What if LSTM loses to LightGBM? Report the result honestly and compare accuracy, stability, runtime, and complexity.
- What are limitations? Simulated benchmark data, distribution shift, finite calibration data, within-engine dependence, and split sensitivity.

15 | Final Submission Checklist

- □ Dataset subset, target convention, feature list, sequence length, and endpoint rule documented.
- □ Temporal analysis tables and plots saved.
- □ Windowing, engine grouping, and leakage tests pass.
- □ LSTM architecture and selected hyperparameters recorded.
- □ Training/validation curves and error diagnostics saved.
- □ MAE, RMSE, and agreed NASA Score calculated with shared evaluator.
- □ Calibration split, quantile method, nominal level, and assumptions documented.
- □ Coverage and Mean Width reported together.
- □ Classical comparison uses same split, endpoint protocol, and metric implementation.
- □ Predictions, checkpoint, scaler, configuration, and environment details saved.
- □ Tables distinguish measured results from planned settings; no invented scores.
- □ Another teammate can reproduce outputs using written instructions.

16 | Suggested Final Report Structure

1. Objective and dataset subset · 2. Temporal data audit · 3. RUL target and sequence construction · 4. Leakage-aware split and preprocessing · 5. LSTM architecture and training · 6. Hyperparameter selection · 7. Point-prediction results · 8. Conformal uncertainty method · 9. Coverage and interval width · 10. Classical-model comparison · 11. Error analysis and limitations · 12. Reproducibility and artifact inventory.

Reference and scope note

This guide expands the supplied team task division into an actionable implementation blueprint. Dataset-specific decisions—including subset, RUL cap, split, metric formula, and availability of test labels—must be confirmed with the team. Illustrations are conceptual and are not experimental output.